This is a comprehensive guideline to help your team successfully build and submit this **Microservices Hotel Management System**.

Since you are beginners using **Java Spring Boot** and **AWS RDS (Cloud Database)**, follow this exact roadmap to avoid confusion.

1. Architecture Overview

- **Structure:** 7 Independent Spring Boot Applications.
- **Database:** 1 Shared AWS RDS MySQL Database (containing 7 different sets of tables).
- **Communication:** REST APIs (Services talk to each other using HTTP requests).
- **Frontend:** React/Angular (Connecting to these 7 backends).

Phase 1: The Setup (Do this together)

Before anyone writes code, sit down as a group and set up the foundation.

- **Create the AWS Database:**
  - One person creates the **AWS RDS MySQL** instance (Free Tier).
  - **Action:** Copy the Endpoint URL, Username, and Password.
  - **Share:** Give these credentials to every team member. Everyone must use the *exact same* credentials in their application.properties.
- **Assign Ports:**
  - Ensure no two services try to run on the same port.
  - **Service 1 (Employee):** 8081
  - **Service 2 (Room):** 8082
  - **Service 3 (Reservation):** 8083
  - **Service 4 (Inventory):** 8084
  - **Service 5 (Restaurant):** 8085
  - **Service 6 (Kitchen):** 8086
  - **Service 7 (Event):** 8087
- **Standardize Project Creation:**
  - Go to **start.spring.io**.
  - **Project:** Maven.
  - **Language:** Java.
  - **Spring Boot:** 3.x.x.
  - **Dependencies (Add these for ALL 7 projects):**
    - Spring Web
    - Spring Data JPA
    - MySQL Driver
    - Lombok
  - **Generate:** Download the 7 zips and distribute them to the team members

responsible for them.

Phase 2: Development Strategy (Divide and Conquer)

Split the team into pairs or individuals.

- **Member A:** Starts with **Service 1 (Employee)**. It is the easiest. Then does **Service 7 (Event)**.
- **Member B:** Takes **Service 2 (Room)** and **Service 3 (Reservation)**. *Reason: These two talk to each other, so one person should handle the logic to avoid confusion.*
- **Member C:** Takes **Service 5 (Restaurant)** and **Service 6 (Kitchen)**. *Reason: These two are also linked.*
- **Member D:** Takes **Service 4 (Inventory)** and works on the **Frontend/UI**.

Phase 3: The Coding Workflow (Repeat for every service)

For every single service, follow this **4-Step Cycle**:

1. **The Entity (Database Table):**
   - Create the Java class (e.g., Room.java).
   - Add @Entity, @Table(name="unique_name"), and @Id.
   - *Tip:* Always define the table name explicitly to avoid conflicts in the shared database.
2. **The Repository (Data Access):**
   - Create the interface (e.g., RoomRepository).
   - Extend JpaRepository.
3. **The Service (Business Logic):**
   - Create the class (e.g., RoomService).
   - Write methods for Create, Read, Update, Delete (CRUD).
4. **The Controller (API Endpoints):**
   - Create the class (e.g., RoomController).
   - Add @RestController, @RequestMapping, and @CrossOrigin (Crucial for frontend connection).

Phase 4: Connecting Services (The "Micro" part)

You have two main connection points in this project. Handle these carefully.

Connection 1: Reservation -> Room

- **Scenario:** When booking a room, check if it exists.
- **Tool:** Use RestTemplate.
- **Code:** Inside ReservationService, call

restTemplate.getForObject("http://localhost:8082/api/rooms/" + id, ...)

Connection 2: Waiter -> Kitchen

- **Scenario:** Waiter places an order, it appears on Chef's screen.
- **Method:** This is actually just a database save!
- **Flow:**
  1. Frontend sends JSON to **KitchenService** (Port 8086).
  2. **KitchenService** saves it to kitchen_orders table.
  3. Chef's Frontend polls **KitchenService** (GET /active) to see the new order.

Phase 5: Testing & Debugging

How to test without a Frontend:

Use **Postman**. It is a tool that lets you send HTTP requests to your Java code.

- **Test 1 (Connectivity):** Run the app. Check the AWS RDS. Did the table appear? If yes, your application.properties is correct.
- **Test 2 (CRUD):** Can you save data (POST)? Can you get it back (GET)?
- **Test 3 (Integration):** Run Service 2 AND Service 3 together. Send a booking request to Service 3. Does it fail if the Room ID doesn't exist in Service 2?

Common Mistakes to Avoid (Read this carefully!)

1. **The "Localhost" Trap:**
   - When deploying or running on different computers, localhost in your Java code won't work for Service-to-Service communication.
   - *Fix:* For now, since you are running everything on one laptop for the demo, localhost is fine. If running on different laptops, you need to use IP addresses.
2. **CORS Errors:**
   - If your React frontend fails to fetch data, you likely forgot @CrossOrigin on the Controller.
3. **Port Conflicts:**
   - If you see "Port 8080 is already in use", check application.properties. You likely forgot to set server.port=808X.
4. **Database Table Names:**
   - Do NOT name a table user or order. These are reserved keywords in SQL. Use app_user or kitchen_orders instead.

Final Checklist for Submission

- [ ] All 7 Services run on unique ports (8081-8087).

- [ ] All 7 Services connect to the **same** AWS Endpoint.
- [ ] All 7 Services have @CrossOrigin enabled.
- [ ] You have a README.md file listing the endpoints for the lecturer (e.g., GET http://localhost:8081/api/employees).
- [ ] You have a Postman Collection exported to show you tested the APIs.

Recommendation: Which Database to Use?

Since you are using AWS and are beginners, I strongly recommend:

AWS RDS (Relational Database Service) for MySQL.

- **Why AWS RDS?** It manages the database for you. You don't need to install MySQL on your laptop. You just get a URL (Endpoint) and connect to it.
- **Why MySQL?** It is the most popular, has the best tutorials for Spring Boot, and is **Free Tier eligible** on AWS (so it won't cost you money if you stay within limits).

The New "Shared Database" Architecture

- **Old Plan:** 7 Services -> 7 Databases (db_employee, db_room, etc.)
- **New Plan:** 7 Services -> **1 AWS RDS Database** (let's call it hotel_db).
  - **Service 1** reads/writes to the employees table.
  - **Service 2** reads/writes to the rooms table.
  - **Service 3** reads/writes to the reservations table.

Warning for Beginners: Since everyone shares the database, **be careful with table names**.

Do not name a table just user in Service 1 and user in Service 2. Give them distinct names like employees and guests.

Service 1: Employee Management Service (AWS Edition)

Here is your updated technical plan. The Java code remains largely the same, but the **Configuration** changes significantly to connect to the Cloud.

1. AWS Setup (Do this first!)

Before writing code, you need the cloud database running.

2. **Log in to AWS Console.**
3. Search for **RDS** (Managed Relational Database Service).
4. Click **"Create database"**.
5. **Choose a creation method:** Standard Create.
6. **Engine options:** MySQL.
7. **Templates:** Select **Free tier** (Important!).
8. **Settings:**
   - **DB Instance Identifier:** hotel-project-db
   - **Master username:** admin (or whatever you choose).
   - **Master password:** Choose a strong password (write it down!).
9. **Connectivity:**
   - **Public access: Select "Yes"** (Otherwise your laptop cannot connect to it).
   - **VPC Security Group:** Create new. (You might need to add an "Inbound Rule" later to allow your laptop's IP address to access port 3306).
10. Click **Create database**.
11. Wait 5-10 minutes. Once it says "Available", click on the database name and copy the **Endpoint** (It looks like hotel-project-db.cw34....us-east-1.rds.amazonaws.com). This is your "Host".

2. Technical Setup

- **Project Name:** employee-service
- **Port:** 8081
- **Dependencies:** Spring Web, Spring Data JPA, **MySQL Driver**, Lombok.

3. Database Design (Single Table)

In your big hotel_db, this service owns one table.

Table: employees

Column	Type
id	Long (PK, Auto Inc)
name	String
role	String (ADMIN, STAFF)
email	String

4. Coding Plan

Step 1: Entity (Employee.java)

Java

```java
@Entity
@Table(name = "employees") // Explicitly name your table
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String role;
    private String email;
    // Getters and Setters (use @Data from Lombok)
}
```

Step 2: Repository (EmployeeRepository.java)

Java

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}
```

Step 3: Service (EmployeeService.java)

Java

```
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository repository;

    public Employee saveEmployee(Employee emp) {
        return repository.save(emp);
    }
    // Add other CRUD methods here
}
```

Step 4: Controller (EmployeeController.java)

Java

```
@RestController
@RequestMapping("/api/employees")
@CrossOrigin // Allow frontend access
public class EmployeeController {
    @Autowired
    private EmployeeService service;

    @PostMapping
    public Employee create(@RequestBody Employee emp) {
        return service.saveEmployee(emp);
    }
}
```

5. Configuration (The Critical AWS Part)

This is where the magic happens. You are no longer connecting to localhost.

File: src/main/resources/application.properties

Properties

```
# 1. Server Configuration
server.port=8081
spring.application.name=employee-service

# 2. AWS RDS Database Configuration
# REPLACE THE URL BELOW WITH YOUR AWS ENDPOINT
spring.datasource.url=jdbc:mysql://hotel-project-db.cx7....us-east-1.rds.amazonaws.com:3306/hotel_db?createDatabaseIfNotExist=true
spring.datasource.username=admin
spring.datasource.password=YOUR_AWS_PASSWORD

# 3. JPA Settings (To create tables automatically)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

Important Note for Beginners:

The part ?createDatabaseIfNotExist=true at the end of the URL is helpful. It tells MySQL to create the hotel_db schema if you forgot to create it manually in AWS.

Next Steps for You

5. **Go to AWS** and create the RDS MySQL database (Free Tier).
6. **Copy the Endpoint.**
7. **Update your application.properties** with that endpoint.
8. **Run Service 1.**
9. Check the logs. If it says "Started EmployeeApplication", it successfully connected to the cloud!

Service 2: Room Management Service

Goal: To manage the hotel rooms. This service will connect to the **same AWS database** as Service 1 but will create and manage its own table called rooms.

---

1. Technical Setup

- **Project Name:** room-service
- **Server Port:** 8082 (Service 1 is on 8081).
- **Database:** The shared AWS RDS MySQL instance.
- **Dependencies:**
  - Spring Web
  - Spring Data JPA
  - **MySQL Driver** (Removed PostgreSQL driver)
  - Lombok

---

2. Database Design (The rooms Table)

In your shared AWS database (hotel_db), this service owns the rooms table.

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique ID (1, 2, 3...).
room_number	VARCHAR(10)	"101", "205", "Suite-A".

type	VARCHAR(50)	"Single", "Double", "Deluxe".
price	DOUBLE	Price per night (e.g., 5000.00).
status	VARCHAR(20)	**Crucial:** "AVAILABLE", "OCCUPIED", "MAINTENANCE".

---

3. Coding Plan (Step-by-Step)

Step 1: The Entity (Room.java)

You must explicitly name the table rooms so it doesn't conflict with other services.

Java

```java
package com.nsbm.room.entity;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
@Table(name = "rooms") // This creates a "rooms" table in your AWS DB
public class Room {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String roomNumber;
    private String type;
    private Double price;

    // We use String for simplicity, but Enum is better for advanced users
```

```
    private String status;
}
```

Step 2: The Repository (RoomRepository.java)

Java

```java
package com.nsbm.room.repository;

import com.nsbm.room.entity.Room;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface RoomRepository extends JpaRepository<Room, Long> {
    // Custom method to find only clean/empty rooms
    List<Room> findByStatus(String status);
}
```

Step 3: The Service (RoomService.java)

Java

```java
package com.nsbm.room.service;

import com.nsbm.room.entity.Room;
import com.nsbm.room.repository.RoomRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class RoomService {
    @Autowired
    private RoomRepository repository;
```

```java
    // 1. Add a new room
    public Room saveRoom(Room room) {
        return repository.save(room);
    }

    // 2. Get all rooms
    public List<Room> getAllRooms() {
        return repository.findAll();
    }

    // 3. Get only available rooms (For the booking website)
    public List<Room> getAvailableRooms() {
        return repository.findByStatus("AVAILABLE");
    }

    // 4. Update status (Used when a guest checks in/out)
    public Room updateStatus(Long id, String newStatus) {
        Room room = repository.findById(id).orElse(null);
        if (room != null) {
            room.setStatus(newStatus);
            return repository.save(room);
        }
        return null;
    }
}
```

Step 4: The Controller (RoomController.java)

Java

```java
package com.nsbm.room.controller;

import com.nsbm.room.entity.Room;
import com.nsbm.room.service.RoomService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

```java
@RestController
@RequestMapping("/api/rooms")
@CrossOrigin // Allow React/Angular to access this
public class RoomController {
    @Autowired
    private RoomService service;

    @PostMapping
    public Room createRoom(@RequestBody Room room) {
        return service.saveRoom(room);
    }

    @GetMapping
    public List<Room> getAll() {
        return service.getAllRooms();
    }

    @GetMapping("/available")
    public List<Room> getAvailable() {
        return service.getAvailableRooms();
    }

    @PatchMapping("/{id}/status")
    public Room updateStatus(@PathVariable Long id, @RequestParam String status) {
        return service.updateStatus(id, status);
    }
}
```

---

4. Configuration (Connecting to Cloud)

This application.properties file will look **almost identical** to Service 1.

Why? Because they connect to the *same* database URL. The only difference is the server.port and application.name.

File: src/main/resources/application.properties

```
Properties
```

```
# 1. Server Configuration
server.port=8082
spring.application.name=room-service

# 2. AWS RDS Database Configuration (SAME AS SERVICE 1)
# Paste the exact same AWS Endpoint here
spring.datasource.url=jdbc:mysql://hotel-project-db.cx7....us-east-1.rds.amazonaws.com:3306/hotel_db
spring.datasource.username=admin
spring.datasource.password=YOUR_AWS_PASSWORD

# 3. JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

5. How to Test (The "Magic" Moment)

- **Run Service 1 (Employee):** It starts on Port 8081.
- **Run Service 2 (Room):** It starts on Port 8082.
- **Check AWS:** If you connect to your AWS database using a tool (like MySQL Workbench or DBeaver), you will now see **two tables** inside hotel_db:
  - employees (created by Service 1)
  - rooms (created by Service 2)

This proves that your microservices are successfully sharing the cloud database!

Service 3: Reservation Management Service (AWS Cloud Edition).

This service is the "Bridge." Even though all services share one big AWS database, **Service 3 must not touch the rooms table directly.** That table belongs to Service 2.

Instead, Service 3 will:

- **Ask Service 2** (via HTTP): "Is Room 101 free?"
- **Save the Booking** in its own reservations table.
- **Tell Service 2** (via HTTP): "Mark Room 101 as OCCUPIED."

Service 3: Reservation Management Service

- **Project Name:** reservation-service
- **Server Port:** 8083 (Service 1: 8081, Service 2: 8082).
- **Database:** The shared AWS RDS MySQL instance.
- **Dependencies:** Spring Web, Spring Data JPA, MySQL Driver, Lombok.

1. Database Design (The reservations Table)

In your AWS hotel_db, this service owns the reservations table.

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique Booking ID.
room_id	BIGINT	The ID of the room (Reference to Service 2).
customer_name	VARCHAR(255)	"John Doe".
check_in	DATE	"2023-12-01".
check_out	DATE	"2023-12-05".
status	VARCHAR(20)	"CONFIRMED", "CANCELLED".

2. Coding Plan (Step-by-Step)

Step 1: The Entity (Reservation.java)

Java

```java
package com.nsbm.reservation.entity;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDate;
```

```java
@Entity
@Data
@Table(name = "reservations") // Creates the "reservations" table in AWS
public class Reservation {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long roomId; // Just the ID, not the full Room object
    private String customerName;
    private LocalDate checkIn;
    private LocalDate checkOut;
    private String status; // "CONFIRMED"
}
```

Step 2: The DTO (Data Transfer Object)

Since this service talks to Service 2, it needs a temporary class to hold the Room data it receives. This class **is not** an Entity (no @Entity) and does not go into the database.

Java

```java
package com.nsbm.reservation.dto;

import lombok.Data;

// This matches the JSON response from Service 2
@Data
public class RoomDTO {
    private Long id;
    private String roomNumber;
    private String status; // We need to check if this is "AVAILABLE"
    private Double price;
}
```

Step 3: The Service (ReservationService.java)

This is where the **Microservice Communication** happens. We use RestTemplate to make the HTTP call.

Java

```java
package com.nsbm.reservation.service;

import com.nsbm.reservation.entity.Reservation;
import com.nsbm.reservation.dto.RoomDTO;
import com.nsbm.reservation.repository.ReservationRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class ReservationService {
    @Autowired
    private ReservationRepository repository;

    @Autowired
    private RestTemplate restTemplate; // Tool to call other APIs

    public Reservation createReservation(Reservation res) {
        // 1. COMMUNICATE: Call Service 2 to check Room status
        String roomServiceUrl = "http://localhost:8082/api/rooms/" + res.getRoomId();
        RoomDTO room = restTemplate.getForObject(roomServiceUrl, RoomDTO.class);

        // 2. LOGIC: Is the room valid and available?
        if (room != null && "AVAILABLE".equals(room.getStatus())) {

            // A. Save the reservation
            res.setStatus("CONFIRMED");
            Reservation savedRes = repository.save(res);

            // B. COMMUNICATE: Tell Service 2 to lock the room
            String updateUrl = "http://localhost:8082/api/rooms/" + res.getRoomId() +
"/status?status=OCCUPIED";
            restTemplate.patchForObject(updateUrl, null, RoomDTO.class);

            return savedRes;
        } else {
```

```
        throw new RuntimeException("Room is not available!");
    }
  }
}
```

Important Config for RestTemplate:

You must create a "Bean" for RestTemplate in your main application class for @Autowired to work.

Java

```
@Bean
public RestTemplate restTemplate() {
  return new RestTemplate();
}
```

Step 4: The Controller (ReservationController.java)

Java

```
package com.nsbm.reservation.controller;

import com.nsbm.reservation.entity.Reservation;
import com.nsbm.reservation.service.ReservationService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/reservations")
@CrossOrigin
public class ReservationController {
  @Autowired
  private ReservationService service;

  @PostMapping
```

```
    public Reservation bookRoom(@RequestBody Reservation res) {
        return service.createReservation(res);
    }
}
```

---

3. Configuration (application.properties)

Same AWS URL as before, just a different port.

```
Properties
```

```
# 1. Server Configuration
server.port=8083
spring.application.name=reservation-service

# 2. AWS RDS Database Configuration
spring.datasource.url=jdbc:mysql://hotel-project-db.cx7....us-east-1.rds.amazonaws.com:3306/hotel_db
spring.datasource.username=admin
spring.datasource.password=YOUR_AWS_PASSWORD

# 3. JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

---

4. How to Test the Integration

This is the cool part. You are testing **Service 3 talking to Service 2**.

- **Start Service 2 (Room Service):** Port 8082.
  - Ensure you have a Room with ID 1 and status "AVAILABLE".
- **Start Service 3 (Reservation Service):** Port 8083.
- **Open Postman.**
- **Send POST Request** to http://localhost:8083/api/reservations:
  JSON
  {

```
  "roomId": 1,
  "customerName": "Kamal Perera",
  "checkIn": "2024-01-01",
  "checkOut": "2024-01-05"
}
```

- **Check Results:**
  - **Success:** You get a JSON response with "status": "CONFIRMED".
  - **Check AWS DB:** Look at the reservations table -> A new row exists.
  - **Check AWS DB (Room Table):** Look at Room 1 -> Status should now be OCCUPIED.

If that works, you have successfully built a communicating microservice system!

Service 4: Inventory Management Service (AWS Cloud Edition).

This service is the "Storekeeper." It connects to the same shared AWS database but manages its own table called inventory.

Service 4: Inventory Management Service

Goal: To track the hotel's stock (food ingredients, cleaning supplies, linens). It allows staff to add items and update quantities when they are used.

---

1. Technical Setup

- **Project Name:** inventory-service
- **Server Port:** 8084 (Service 3 is on 8083).
- **Database:** The shared AWS RDS MySQL instance.
- **Dependencies:**
  - Spring Web
  - Spring Data JPA
  - **MySQL Driver** (Standardized for all services)
  - Lombok

---

2. Database Design (The inventory Table)

In your shared AWS database (hotel_db), this service owns the inventory table.

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique ID.
name	VARCHAR(255)	"Tomato", "Bath Towel", "Soap".
category	VARCHAR(50)	"KITCHEN", "HOUSEKEEPING", "MAINTENANCE".
quantity	INT	Current stock level (e.g., 50).
unit	VARCHAR(20)	"kg", "liters", "pcs".

---

3. Coding Plan (Step-by-Step)

Step 1: The Entity (InventoryItem.java)

Java

```java
package com.nsbm.inventory.entity;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
@Table(name = "inventory") // Creates the "inventory" table in AWS
public class InventoryItem {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private int quantity;
    private String unit;
}
```

Step 2: The Repository (InventoryRepository.java)

Java

```java
package com.nsbm.inventory.repository;

import com.nsbm.inventory.entity.InventoryItem;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface InventoryRepository extends JpaRepository<InventoryItem, Long> {
    // Custom method to find items by category (e.g., "Give me all KITCHEN items")
    List<InventoryItem> findByCategory(String category);
```

}

Step 3: The Service (InventoryService.java)

Java

```java
package com.nsbm.inventory.service;

import com.nsbm.inventory.entity.InventoryItem;
import com.nsbm.inventory.repository.InventoryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class InventoryService {
    @Autowired
    private InventoryRepository repository;

    // 1. Add new item
    public InventoryItem saveItem(InventoryItem item) {
        return repository.save(item);
    }

    // 2. Get all items
    public List<InventoryItem> getAllItems() {
        return repository.findAll();
    }

    // 3. Update Stock (Add or Remove)
    public InventoryItem updateStock(Long id, int quantityChange) {
        InventoryItem item = repository.findById(id).orElse(null);
        if (item != null) {
            int newQuantity = item.getQuantity() + quantityChange;
            if (newQuantity < 0) {
                throw new RuntimeException("Not enough stock!");
            }
            item.setQuantity(newQuantity);
```

```
        return repository.save(item);
    }
    return null;
  }
}
```

Step 4: The Controller (InventoryController.java)

Java

```java
package com.nsbm.inventory.controller;

import com.nsbm.inventory.entity.InventoryItem;
import com.nsbm.inventory.service.InventoryService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/inventory")
@CrossOrigin // Important for Frontend
public class InventoryController {
  @Autowired
  private InventoryService service;

  @PostMapping
  public InventoryItem create(@RequestBody InventoryItem item) {
    return service.saveItem(item);
  }

  @GetMapping
  public List<InventoryItem> getAll() {
    return service.getAllItems();
  }

  // Example Usage: PUT /api/inventory/1/stock?change=-5 (Removes 5 items)
  @PutMapping("/{id}/stock")
  public InventoryItem updateStock(@PathVariable Long id, @RequestParam int change) {
    return service.updateStock(id, change);
```

```
    }
}
```

---

4. Configuration (application.properties)

File: src/main/resources/application.properties

```
Properties
```

```
# 1. Server Configuration
server.port=8084
spring.application.name=inventory-service

# 2. AWS RDS Database Configuration (SAME AS OTHERS)
spring.datasource.url=jdbc:mysql://hotel-project-db.cx7....us-east-1.rds.amazonaws.com:3306/hotel_db
spring.datasource.username=admin
spring.datasource.password=YOUR_AWS_PASSWORD

# 3. JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

---

5. How to Test

- **Run Service 4:** It starts on Port 8084.
- **Open Postman.**
- **Create an Item (POST):**
  - URL: http://localhost:8084/api/inventory
  - Body:
    JSON
    ```
    {
      "name": "Rice",
      "category": "KITCHEN",
      "quantity": 100,
      "unit": "kg"
    ```

```
}
```

- **Update Stock (PUT):**
  - URL: http://localhost:8084/api/inventory/1/stock?change=-10
  - Result: The quantity in the database should drop to **90**.
- **Check AWS DB:** You will now see the inventory table next to employees, rooms, and reservations.

Service 5: Restaurant Management Service (AWS Cloud Edition).

This service manages the dining area. It connects to the same AWS database but owns two specific tables: one for the **Menu** and one for the **Tables**.

Service 5: Restaurant Management Service

- **Project Name:** restaurant-service
- **Server Port:** 8085 (Service 4 is on 8084).
- **Database:** The shared AWS RDS MySQL instance.
- **Dependencies:** Spring Web, Spring Data JPA, MySQL Driver, Lombok.

---

1. Database Design (Two Tables)

In your shared AWS database (hotel_db), this service creates these two tables.

Table 1: menu_items

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique ID.
name	VARCHAR(255)	"Chicken Fried Rice".
price	DOUBLE	1500.00
available	BOOLEAN	true (In stock) / false (Sold out).

Table 2: restaurant_tables

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique ID.

table_number	INT	The label on the table (e.g., 5).
capacity	INT	Seats (e.g., 4).
is_occupied	BOOLEAN	true (Guests seated) / false (Empty).

---

2. Coding Plan (Step-by-Step)

Step 1: The Entities

MenuItem.java

Java

```java
package com.nsbm.restaurant.entity;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
@Table(name = "menu_items")
public class MenuItem {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double price;
    private boolean available; // True if kitchen has ingredients
}
```

RestaurantTable.java

Java

```java
package com.nsbm.restaurant.entity;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
@Table(name = "restaurant_tables")
public class RestaurantTable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private int tableNumber;
    private int capacity;
    private boolean isOccupied; // True if customers are sitting here
}
```

Step 2: The Repositories

You need two interfaces in the same package.

Java

```java
public interface MenuItemRepository extends JpaRepository<MenuItem, Long> {
    // Find all food that is currently available
    List<MenuItem> findByAvailableTrue();
}

public interface RestaurantTableRepository extends JpaRepository<RestaurantTable, Long> {
    // Find all empty tables
    List<RestaurantTable> findByIsOccupiedFalse();
}
```

Step 3: The Service (RestaurantService.java)

Java

```java
@Service
public class RestaurantService {
    @Autowired
    private MenuItemRepository menuRepo;
    @Autowired
    private RestaurantTableRepository tableRepo;

    // --- MENU LOGIC ---
    public MenuItem addDish(MenuItem item) {
        return menuRepo.save(item);
    }

    public List<MenuItem> getMenu() {
        return menuRepo.findByAvailableTrue(); // Only show what we can cook
    }

    // --- TABLE LOGIC ---
    public RestaurantTable addTable(RestaurantTable table) {
        return tableRepo.save(table);
    }

    public RestaurantTable seatGuests(Long tableId) {
        RestaurantTable table = tableRepo.findById(tableId).orElse(null);
        if (table != null) {
            table.setOccupied(true);
            return tableRepo.save(table);
        }
        return null;
    }

    public RestaurantTable freeTable(Long tableId) {
        RestaurantTable table = tableRepo.findById(tableId).orElse(null);
        if (table != null) {
            table.setOccupied(false);
```

```
        return tableRepo.save(table);
    }
    return null;
  }
}
```

Step 4: The Controller (RestaurantController.java)

Java

```java
@RestController
@RequestMapping("/api/restaurant")
@CrossOrigin
public class RestaurantController {
  @Autowired
  private RestaurantService service;

  // --- MENU ENDPOINTS ---
  @PostMapping("/menu")
  public MenuItem addDish(@RequestBody MenuItem item) {
    return service.addDish(item);
  }

  @GetMapping("/menu")
  public List<MenuItem> getMenu() {
    return service.getMenu();
  }

  // --- TABLE ENDPOINTS ---
  @PostMapping("/tables")
  public RestaurantTable addTable(@RequestBody RestaurantTable table) {
    return service.addTable(table);
  }

  @PutMapping("/tables/{id}/occupy")
  public RestaurantTable occupyTable(@PathVariable Long id) {
    return service.seatGuests(id);
  }
```

```
    @PutMapping("/tables/{id}/free")
    public RestaurantTable freeTable(@PathVariable Long id) {
        return service.freeTable(id);
    }
}
```

---

3. Configuration (application.properties)

File: src/main/resources/application.properties

Properties

```
# 1. Server Configuration
server.port=8085
spring.application.name=restaurant-service

# 2. AWS RDS Database Configuration (SAME AS OTHERS)
spring.datasource.url=jdbc:mysql://hotel-project-db.cx7....us-east-1.rds.amazonaws.com:3306/hotel_db
spring.datasource.username=admin
spring.datasource.password=YOUR_AWS_PASSWORD

# 3. JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

---

4. How to Test

- **Run Service 5:** It starts on Port 8085.
- **Open Postman.**
- **Add a Dish (POST):**
  - URL: http://localhost:8085/api/restaurant/menu
  - Body: {"name": "Burger", "price": 850.00, "available": true}
- **Add a Table (POST):**
  - URL: http://localhost:8085/api/restaurant/tables
  - Body: {"tableNumber": 1, "capacity": 4, "isOccupied": false}

- **Seat Guests (PUT):**
  - URL: http://localhost:8085/api/restaurant/tables/1/occupy
  - Result: isOccupied becomes true.
- **Check AWS DB:** You will now see menu_items and restaurant_tables in the hotel_db.

Service 6: Kitchen Management Service (AWS Cloud Edition).

This service acts as the **"Chef's Display."** It connects to the same AWS database but manages its own table called kitchen_orders.

Service 6: Kitchen Management Service

- **Project Name:** kitchen-service
- **Server Port:** 8086 (Service 5 is on 8085).
- **Database:** The shared AWS RDS MySQL instance.
- **Dependencies:** Spring Web, Spring Data JPA, MySQL Driver, Lombok.

---

1. Database Design (The kitchen_orders Table)

In your shared AWS database (hotel_db), this service owns the kitchen_orders table.

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique Order ID.
table_id	BIGINT	The table number (reference to Service 5).
menu_item_id	BIGINT	The dish ID (reference to Service 5).

quantity	INT	How many (e.g., 2).
status	VARCHAR(20)	"RECEIVED", "PREPARING", "READY".
order_time	DATETIME	When the waiter punched it in.

---

2. Coding Plan (Step-by-Step)

Step 1: The Entity (KitchenOrder.java)

Java

```java
package com.nsbm.kitchen.entity;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Data
@Table(name = "kitchen_orders") // Creates "kitchen_orders" table in AWS
public class KitchenOrder {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long tableId;
    private Long menuItemId;
    private int quantity;
    private String status; // "RECEIVED", "PREPARING", "READY"
    private LocalDateTime orderTime;

    @PrePersist
```

```
    public void prePersist() {
        this.orderTime = LocalDateTime.now(); // Auto-set time on creation
    }
}
```

Step 2: The Repository (KitchenRepository.java)

Java

```
package com.nsbm.kitchen.repository;

import com.nsbm.kitchen.entity.KitchenOrder;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface KitchenRepository extends JpaRepository<KitchenOrder, Long> {
    // Chef needs to see ONLY active orders
    List<KitchenOrder> findByStatus(String status);
}
```

Step 3: The Service (KitchenService.java)

Java

```
package com.nsbm.kitchen.service;

import com.nsbm.kitchen.entity.KitchenOrder;
import com.nsbm.kitchen.repository.KitchenRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class KitchenService {
    @Autowired
```

```java
    private KitchenRepository repository;

    // 1. Waiter sends a new order
    public KitchenOrder createOrder(KitchenOrder order) {
        order.setStatus("RECEIVED");
        return repository.save(order);
    }

    // 2. Chef views active orders
    public List<KitchenOrder> getActiveOrders() {
        // In a real app, you'd combine these lists
        return repository.findByStatus("RECEIVED");
    }

    // 3. Chef updates status (e.g., Starts cooking)
    public KitchenOrder updateStatus(Long id, String newStatus) {
        KitchenOrder order = repository.findById(id).orElse(null);
        if (order != null) {
            order.setStatus(newStatus);
            return repository.save(order);
        }
        return null;
    }
}
```

Step 4: The Controller (KitchenController.java)

Java

```java
package com.nsbm.kitchen.controller;

import com.nsbm.kitchen.entity.KitchenOrder;
import com.nsbm.kitchen.service.KitchenService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
```

```java
@RequestMapping("/api/kitchen")
@CrossOrigin
public class KitchenController {
    @Autowired
    private KitchenService service;

    @PostMapping("/orders")
    public KitchenOrder createOrder(@RequestBody KitchenOrder order) {
        return service.createOrder(order);
    }

    @GetMapping("/orders/active")
    public List<KitchenOrder> getActiveOrders() {
        return service.getActiveOrders();
    }

    @PatchMapping("/orders/{id}/status")
    public KitchenOrder updateStatus(@PathVariable Long id, @RequestParam String status) {
        return service.updateStatus(id, status);
    }
}
```

---

3. Configuration (application.properties)

File: src/main/resources/application.properties

Properties

```properties
# 1. Server Configuration
server.port=8086
spring.application.name=kitchen-service

# 2. AWS RDS Database Configuration (SAME AS OTHERS)
spring.datasource.url=jdbc:mysql://hotel-project-db.cx7....us-east-1.rds.amazonaws.com:3306/hotel_db
spring.datasource.username=admin
spring.datasource.password=YOUR_AWS_PASSWORD
```

```
# 3. JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

---

4. How to Test

- **Run Service 6:** It starts on Port 8086.
- **Open Postman.**
- **Waiter Sends Order (POST):**
  - URL: http://localhost:8086/api/kitchen/orders
  - Body:
    ```JSON
    {
      "tableId": 5,
      "menuItemId": 10,
      "quantity": 2
    }
    ```
  - Result: Status is "RECEIVED".
- **Chef Updates Status (PATCH):**
  - URL: http://localhost:8086/api/kitchen/orders/1/status?status=PREPARING
  - Result: Status changes to "PREPARING".
- **Check AWS DB:** Look for the kitchen_orders table.

Service 7: Event Management Service (AWS Cloud Edition).

This service manages the "Big Picture" items: Banquet Halls, Conference Rooms, and Wedding Gardens. It connects to the same AWS database but owns the venues and events tables.

Service 7: Event Management Service

- **Project Name:** event-service
- **Server Port:** 8087 (Service 6 is on 8086).
- **Database:** The shared AWS RDS MySQL instance.
- **Dependencies:** Spring Web, Spring Data JPA, MySQL Driver, Lombok.

---

1. Database Design (Two Tables)

In your shared AWS database (hotel_db), this service creates these two tables.

Table 1: venues (The Halls)

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique ID.
name	VARCHAR(255)	"Grand Ballroom", "Garden Side".
capacity	INT	Max people (e.g., 500).
price_per_hour	DOUBLE	Cost to rent.

Table 2: events (The Bookings)

Column Name	Data Type	Purpose
id	BIGINT (PK)	Unique ID.
venue_id	BIGINT	Which hall? (FK to venues).
customer_name	VARCHAR(255)	Who booked it?
event_date	DATE	When is it?
attendees	INT	How many guests?
status	VARCHAR(20)	"CONFIRMED", "PENDING".

---

2. Coding Plan (Step-by-Step)

Step 1: The Entities

Venue.java

Java

```java
package com.nsbm.event.entity;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
@Table(name = "venues")
public class Venue {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private int capacity;
    private Double pricePerHour;
}
```

Event.java

Java

```java
package com.nsbm.event.entity;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDate;

@Entity
@Data
```

```java
@Table(name = "events")
public class Event {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long venueId; // Simple ID reference
    private String customerName;
    private LocalDate eventDate;
    private int attendees;
    private String status;
}
```

Step 2: The Repositories

You need two interfaces. The EventRepository needs a special method to prevent double-booking.

Java

```java
public interface VenueRepository extends JpaRepository<Venue, Long> {
}

public interface EventRepository extends JpaRepository<Event, Long> {
    // "Find if any event exists at this Venue on this Date"
    List<Event> findByVenueIdAndEventDate(Long venueId, LocalDate date);
}
```

Step 3: The Service (EventService.java)

Java

```java
@Service
public class EventService {
    @Autowired
    private VenueRepository venueRepo;
```

```java
    @Autowired
    private EventRepository eventRepo;

    // 1. Admin adds a new Hall
    public Venue addVenue(Venue venue) {
        return venueRepo.save(venue);
    }

    public List<Venue> getAllVenues() {
        return venueRepo.findAll();
    }

    // 2. Customer Books an Event
    public Event bookEvent(Event event) {
        // Step A: Check Capacity
        Venue venue = venueRepo.findById(event.getVenueId()).orElse(null);
        if (venue == null) {
            throw new RuntimeException("Venue not found!");
        }
        if (event.getAttendees() > venue.getCapacity()) {
            throw new RuntimeException("Too many people for this hall!");
        }

        // Step B: Check Availability (Is it already booked?)
        List<Event> existing = eventRepo.findByVenueIdAndEventDate(event.getVenueId(),
event.getEventDate());
        if (!existing.isEmpty()) {
            throw new RuntimeException("Venue is already booked on this date!");
        }

        // Step C: Save
        event.setStatus("CONFIRMED");
        return eventRepo.save(event);
    }

    public List<Event> getAllEvents() {
        return eventRepo.findAll();
    }
}
```

Step 4: The Controller (EventController.java)

Java

```java
@RestController
@RequestMapping("/api/events")
@CrossOrigin
public class EventController {
    @Autowired
    private EventService service;

    // --- VENUE ENDPOINTS ---
    @PostMapping("/venues")
    public Venue addVenue(@RequestBody Venue venue) {
        return service.addVenue(venue);
    }

    @GetMapping("/venues")
    public List<Venue> getVenues() {
        return service.getAllVenues();
    }

    // --- EVENT ENDPOINTS ---
    @PostMapping("/book")
    public Event bookEvent(@RequestBody Event event) {
        return service.bookEvent(event);
    }

    @GetMapping
    public List<Event> getEvents() {
        return service.getAllEvents();
    }
}
```

---

3. Configuration (application.properties)

File: src/main/resources/application.properties

Properties

```
# 1. Server Configuration
server.port=8087
spring.application.name=event-service

# 2. AWS RDS Database Configuration (SAME AS OTHERS)
spring.datasource.url=jdbc:mysql://hotel-project-db.cx7....us-east-1.rds.amazonaws.com:3306/hotel_db
spring.datasource.username=admin
spring.datasource.password=YOUR_AWS_PASSWORD

# 3. JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

---

4. How to Test

- **Run Service 7:** It starts on Port 8087.
- **Open Postman.**
- **Create a Venue (POST):**
  - URL: http://localhost:8087/api/events/venues
  - Body: {"name": "Grand Ballroom", "capacity": 500, "pricePerHour": 50000.0}
  - Result: ID 1 is created.
- **Book an Event (POST):**
  - URL: http://localhost:8087/api/events/book
  - Body:
    JSON
    ```
    {
      "venueId": 1,
      "customerName": "Nimal Siripala",
      "eventDate": "2024-12-25",
      "attendees": 400
    }
    ```
  - Result: Status "CONFIRMED".
- **Try Double Booking (POST):**
  - Send the exact same request again.
  - Result: **Error 500** ("Venue is already booked on this date!").

- **Check AWS DB:** You will now see venues and events tables.

Project Completion Summary

You now have a complete technical plan for all 7 services using **AWS RDS**.

Service	Port	Table(s) in Shared DB	Purpose
1. Employee	8081	employees	Staff identity & roles.
2. Room	8082	rooms	Managing physical rooms.
3. Reservation	8083	reservations	Booking engine (Talks to Svc 2).
4. Inventory	8084	inventory	Stock tracking.
5. Restaurant	8085	menu_items, restaurant_tables	Dining management.
6. Kitchen	8086	kitchen_orders	Chef's cooking queue.
7. Event	8087	venues, events	Weddings & Halls.

Final Advice for Your Group:

2. **Split the work:** Assign 1-2 services per person.
3. **Share the AWS Credentials:** Make sure everyone has the same application.properties database URL/Password.
4. **Start Coding:** Begin with **Service 1 (Employee)** today to test the connection.

Why do this?

So that "Service 1" runs exactly the same on your laptop as it does on your friend's laptop. You won't have to worry about "It works on my machine but not yours."

Phase 1: Preparation (All Members)

Every group member must do this once:

- **Install Docker Desktop:** Download and install it for Windows/Mac.
- **Start Docker:** Open the app and make sure the engine is running (Green light).
- **Check Terminal:** Open Command Prompt (cmd) and type docker --version. If you see a version number, you are ready.

Phase 2: Add Docker to Each Service (The "Receipt")

You need to tell Docker how to package your Java app. You must add a file named Dockerfile (no extension) to the **root folder** of **EVERY** service (Service 1 to 7).

Steps for Service 1 (Employee Service):

- Open the employee-management-service folder.
- Create a new file named Dockerfile.
- Paste this exact code:

Dockerfile

```
# 1. Start with a Java base image (Like installing Java on a fresh PC)
FROM openjdk:17-jdk-slim

# 2. Copy the jar file from your target folder into the container
# Note: Ensure you run 'mvn clean package' first!
COPY target/*.jar app.jar

# 3. Run the application
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Repeat this step for ALL 7 services.

(Copy and paste this file into room-service, reservation-service, etc.)

Phase 3: The "Magic Button" (Docker Compose)

Instead of running 7 different commands to start your services, we will create **one master script** to run them all.

- Create a **Root Folder** for your whole project (e.g., Hotel-System-Group-Project).
- Move all 7 service folders inside this root folder.
  - Structure:
    - Hotel-System-Group-Project/
      - employee-service/
      - room-service/
      - ... (other 5 folders)
      - **docker-compose.yml** (Create this file here!)
- **Paste this code into docker-compose.yml:**

YAML

```
version: '3.8'
```

```yaml
services:
  # --- Service 1: Employee ---
  employee-service:
    build: ./employee-service  # Look in this folder for the Dockerfile
    ports:
      - "8081:8081"            # Map laptop port 8081 to container port 8081
    container_name: employee-container

  # --- Service 2: Room ---
  room-service:
    build: ./room-service
    ports:
      - "8082:8082"
    container_name: room-container

  # --- Service 3: Reservation ---
  reservation-service:
    build: ./reservation-service
    ports:
      - "8083:8083"
    container_name: reservation-container

  # --- Service 4: Inventory ---
  inventory-service:
    build: ./inventory-service
    ports:
      - "8084:8084"
    container_name: inventory-container

  # --- Service 5: Restaurant ---
  restaurant-service:
    build: ./restaurant-service
    ports:
      - "8085:8085"
    container_name: restaurant-container

  # --- Service 6: Kitchen ---
  kitchen-service:
    build: ./kitchen-service
    ports:
      - "8086:8086"
    container_name: kitchen-container

  # --- Service 7: Event ---
```

```yaml
event-service:
  build: ./event-service
  ports:
    - "8087:8087"
  container_name: event-container
```

Phase 4: The Workflow (How to Run)

Now, whenever you want to work on the project, follow these steps.

Step 1: Build your Java JARs

Docker needs the compiled .jar file. You must generate this first.

- **Open Terminal** in the root folder.
- Run this command for **each** service (or use a script):
  - cd employee-service -> mvn clean package -DskipTests -> cd ..
  - cd room-service -> mvn clean package -DskipTests -> cd ..
  - *(Do this for all 7).*

Step 2: Run Docker Compose

This is the moment of truth.

- In the root folder (where docker-compose.yml is), run:
  Bash
  ```bash
  docker-compose up --build
  ```

What happens next?

1. Docker will read the Dockerfile in every folder.
2. It will wrap every service into a container.
3. It will start all 7 containers at once.
4. You will see logs from all 7 services streaming in one terminal window.

Step 3: Verify

Open your browser or Postman:

- http://localhost:8081/api/employees -> Should respond.
- http://localhost:8082/api/rooms -> Should respond.

Troubleshooting for Beginners

Problem 1: "Connection Refused" between services.

- **Context:** Service 3 (Reservation) tries to call Service 2 (Room).
- **Fix:** When running inside Docker, localhost refers to *the container itself*, not your laptop.
- **Solution:** You must change the URL in your Java code or application.properties.
  - **Old:** http://localhost:8082/api/rooms/...
  - **New (Docker Friendly):** http://room-service:8082/api/rooms/...
  - *Explanation:* Docker creates a "network" where the service name (room-service) becomes the hostname.

Problem 2: "Jar file not found"

- **Fix:** You forgot Step 1 (Building the JARs). You must run mvn clean package before running docker-compose up.

Problem 3: Database Connection Failed

- **Fix:** Ensure your AWS RDS Security Group allows "Anywhere (0.0.0.0/0)" or your specific IP. Since Docker runs inside a virtual network, sometimes AWS blocks it if the security settings are too strict.

Summary Checklist for the Group

1. [ ] AWS RDS Database created & shared.
2. [ ] Dockerfile added to all 7 folders.
3. [ ] docker-compose.yml created in the root folder.
4. [ ] Run mvn clean package for all apps.
5. [ ] Run docker-compose up --build.